Multi-Sensor Multi-Target Tracking System for Harbour Surveillance

1 Background and motivation

Harbour environments are among the most complex and safety-critical scenarios for autonomous maritime systems. Vessels of all sizes, from container ships to small recreational boats, navigate in close proximity within confined waterways alongside fixed infrastructure such as piers, navigational buoys, and moored vessels. Monitoring this environment in real time is essential for collision avoidance, vessel traffic management, intrusion detection, and the safe integration of autonomous surface vessels into port operations.

A single sensor is fundamentally insufficient for harbour surveillance. Every sensing modality has blind spots, range limitations, and failure conditions. A mm-wave radar sees through fog and rain but has limited bearing resolution and cannot classify objects. A stereo camera provides rich visual context but is blocked by obstacles and degrades in low light. The Automatic Identification System (AIS) provides high-accuracy position reports but only for vessels that carry and activate AIS transponders — many small craft do not. Effective surveillance therefore requires the fusion of complementary sensors into a coherent, unified picture of the scene.

This project addresses exactly this problem. Students will design and implement a multi-sensor multi-target tracking system that fuses data from four heterogeneous sources — a land-based mm-wave radar, a land-based stereo camera, an AIS receiver, and a GNSS receiver — mounted at different positions and on different platforms. The system must produce a reliable, real-time estimate of the position, velocity, and heading of all moving objects in the harbour, and must do so robustly in the presence of measurement noise, missed detections, false alarms, and sensor outages.

2 System overview

2.1 Sensor infrastructure

The surveillance system comprises four sensor nodes with the following specifications.

Sensor	Range	FOV	Rate	Platform	Output
mm-wave radar	1 km	360°	0.3 Hz	Land (static)	range + bearing
Stereo camera	0.5 km	180°	0.5 Hz	Land (static)	range + bearing
AIS receiver	5 km	360°	~0.3 Hz	Vessel (moving)	NED position
GNSS receiver	—	—	1 Hz	Vessel (moving)	NED position

The mm-wave radar and stereo camera are mounted at fixed, known positions on land. The inertial reference frame used throughout the project is a local North-East-Down (NED) frame with its origin fixed at the mm-wave radar position. This choice eliminates the need for any coordinate frame transformation for the radar, whose measurements are already expressed relative to its own position. The AIS receiver and GNSS receiver are installed on a surface vessel; the vessel GNSS position is reported directly in the NED frame.

2.2 Fusion centre and coordinate frames

The fusion centre is located on the surface vessel. Three coordinate frames are in play:

- The NED frame (inertial, fixed): origin at the mm-wave radar position. North axis points geographic north, East axis points east, Down axis points toward the Earth centre. All target state vectors are estimated in this frame. Because the origin coincides with the radar, radar measurements need no position offset correction.
- The stereo camera offset: the camera is at a known, fixed NED position $s_c = [N_c \quad E_c]^T$ relative to the NED origin. Camera measurements are (range, bearing) from s_c and must be resolved using this offset.
- The AIS and GNSS: the vessel GNSS position $v(t) = [v_N(t) \quad v_E(t)]^T$ is expressed in the NED frame and is updated at 1 Hz. AIS target reports are provided in the NED frame as absolute positions $[N \quad E]^T$, from which the tracker derives the implied (range, bearing) observation relative to the vessel position. No frame rotation is required for any sensor — all data share the same NED orientation.

The EKF state vector $x = [p_N, p_E, v_N, v_E]^T$ is defined in the NED frame. The measurement function for sensor *i* at NED position $s_i = \left[s_N^i, s_E^i\right]^T$ is $h_i(x) = [\|p - s_i\| \quad atan2(p_E - s_E^i,\ p_N - s_N^i)]^T$. For the radar, $s_i = [0,0]^T$. For the stereo camera and AIS, the offset is non-zero but either constant (camera) or time-varying via GNSS (AIS).

2.3 System architecture

The system follows the track-before-detect architecture with a centralised fusion centre on the vessel. The four functional modules that should be implemented are:

- Coordinate frame manager: it resolves all incoming (range, bearing) measurements against the NED-frame sensor positions, computing $h_i$and its Jacobian H_i for each sensor. No rotation is needed — only position offsets, since all sensors share the NED orientation.
- Sensor-specific EKF tracker: it maintains one EKF instance per confirmed track with a measurement model parameterised by the originating sensor's NED position. Supports the CV motion model and, as an extension, the IMM combining CV and CT models.
- Gating and data association: Mahalanobis-distance gating per track per sensor stream, followed by a data association algorithm of the group's choosing across all active tracks and all gated detections from all sensors simultaneously.
- Track management: full lifecycle from tentative through confirmed, coasting, and deleted, with M-of-N confirmation logic and score-based deletion.

3 Simulation environment

A complete simulation environment is provided as a ready-to-run Google Colab notebook: ***harbour_simulation.ipynb***. Students are expected to understand how it works and to configure it to generate synthetic data for all five mandatory scenarios.

3.1 Structure of the simulation environment

The notebook is organised in 16 sections. The following description maps each section to the underlying design choices students will need to understand in order to use and interpret the simulation output correctly.

Section 0 — Imports Sets up NumPy, Matplotlib, dataclasses, and creates the **harbour_sim_output/** directory automatically in Colab.

Section 1 — Data structures Defines three lightweight data classes. **TargetConfig** holds kinematics and AIS flag. **Measurement** carries one sensor return with a unified schema covering both range-bearing (radar, camera) and NED-position (AIS, GNSS) outputs. **SimulationOutput** bundles everything returned by one run.

Section 2 — Motion model All simulated targets follow a discrete-time constant-velocity (CV) model with state vector $x = [p_N, p_E, v_N, v_E]^T$ in the NED frame. Process noise standard deviation $\sigma_a = 0.05\ m/s^2$ models small unmodelled accelerations due to sea current and engine variation.

Section 3 — Sensor models Each of the four sensors is implemented as a Python class with configurable parameters matching the project specification. The bearing convention throughout is $\phi = atan2(\Delta East, \Delta North)$ — from North, clockwise — consistent with the NED frame and the EKF Jacobian derivation.

- *RadarSensor*: static at the NED origin [0, 0]. Generates (range, bearing) measurements with independent Gaussian noise (σ_r, σ_ϕ). False alarms are drawn from a Poisson process ($\lambda_{\{FA\}}$ expected per scan) with returns uniformly distributed within the 1 km range gate.
- *StereoCameraSensor*: static at configurable NED offset (default [−80, 120] m, boresight 45°). Applies a 180° FOV mask and 500 m range gate before generating (range, bearing) measurements with its own noise parameters. False alarms are uniform within the visible sector.
- *AISSensor*: mounted on the moving vessel. It generates NED position reports at one report per 3s with Gaussian position noise (σ_{AIS}). Only targets configured with has_ais=True generate AIS returns. AIS dropout windows can be injected by passing an ais_dropout=(t_start, t_end) argument to sim.run().
- *GNSSSensor*: provides the vessel own-position at 1 Hz with Gaussian noise (σ_{GNSS}). The vessel position is used by the simulator to compute AIS measurements and is provided to the tracking system via the GNSS measurements in the output JSON.

Section 4 — Vessel trajectory The surface vessel follows a closed polygon patrol route defined by a list of NED waypoints. The vessel moves at constant speed (3 knots) and loops continuously. The vessel's NED position can be inspected using vessel.position(t) and can modify the waypoints or speed to create different patrol patterns.

Section 5 — Simulation engine The HarbourSimulation class is the central simulation engine. Its run() method propagates all target ground-truth states at a fine time step (DT_TRUE = 0.1 s) using the CV motion model, then generates sensor measurements at each sensor's scheduled scan time. The method returns a SimulationOutput object containing:

- ground_truth: a dictionary mapping each target ID to a (T × 4) NumPy array of NED states [p_N, p_E, v_N, v_E] at every time step. NaN entries indicate that the target was not yet active or had left the scene.
- measurements: a time-sorted list of Measurement objects, each carrying the sensor ID, timestamp, true-target ID (−1 for false alarms), and the measurement values (range_m & bearing_rad for radar and camera; north_m & east_m for AIS and GNSS).
- vessel_positions: a (T_GNSS × 2) array of vessel NED positions at each GNSS fix time, providing the time-varying offset needed for AIS measurement model Jacobian computation.

Sections 6–8 cover visualisation, JSON export, and the sensor factory — all tested as part of each scenario run.

Scenarios A–E Produce the ground truth and the measurements for all the testing scenarios.

Section 9 – Load and inspect saved scenario It allows to load and inspect the generated JSON files from the scenarios.

Section 10 — Customisation guide It explains in-notebook how to change sensor noise, disable sensors, inject a coordinated-turn manoeuvre, and fix the random seed for reproducibility — the last point being important for the EKF validation workflow.

3.2 How to run the five scenarios

Each scenario is implemented as a self-contained cell in the notebook. Running a scenario cell produces a four-panel visualisation (2-D NED scene, range over time, bearing over time, detections per scan) and saves a JSON file to the harbour_sim_output/ directory. The five cells may be run independently in any order; each fixes its own random seed to guarantee reproducibility.

The simulation parameters for each scenario match the success criteria defined in Section 5. Students should run the scenarios in order (A through E) as they progress through the project phases, using the JSON output of each scenario to develop and validate the corresponding tracking module. The following parameters are easily configurable within each cell without modifying the simulation engine:

- Sensor noise levels: pass different σ_r, σ_ϕ, etc. arguments to the sensor constructors via make_sensors().
- Detection probability and false alarm rate: pass radar_pd, radar_lfa, camera_pd, camera_lfa arguments to make_sensors().
- Active sensor set: pass active_sensors=['radar','gnss'] to sim.run() to disable the camera and AIS.
- Random seed: set np.random.seed(N) before sim.run() to reproduce any specific realisation.

4 Project structure

The project is structured in four phases of increasing system complexity.

4.1 Phase 1 - State of the art and coordinate framework

4.1.1 T1: State-of-the-art survey

Conduct a structured literature review covering the state of the art in multi-sensor multi-target tracking, with emphasis on maritime and harbour applications. The survey shall address:

- Overview of target tracking architectures: centralised vs decentralised fusion, track-to-track fusion vs measurement-level fusion.
- Data association algorithms: at minimum, Nearest Neighbour (NN), Global Nearest Neighbour (GNN), Probabilistic Data Association (PDA/JPDA), and Multi-Hypothesis Tracking (MHT). The survey must include a comparative analysis of accuracy vs computational cost that will directly motivate the group's implementation choice in T6.
- Multi-sensor EKF fusion: sequential update, joint update, and out-of-sequence measurement handling.
- Maritime-specific considerations: AIS integration, sea clutter, harbour multipath.
- At least one recent deep learning approach to multi-target tracking and a critical comparison with classical model-based methods.

The survey shall conclude with a justified selection of the data association algorithm to be implemented in T6, clearly explaining why it was chosen over alternatives. This selection and its justification form the background of the final poster.

4.1.2 T2: Coordinate frame manager

Implement the coordinate frame manager module — a standalone Python component responsible for computing the measurement function $h_i(x, t)$ and its Jacobian H_i for each sensor, given the NED-frame sensor positions. Because all sensors share the NED orientation, no rotation matrix is involved — only position offsets enter the measurement functions. The module shall:

- Store the known NED-frame offset of the stereo camera relative to the NED origin (mm-wave radar position). The radar itself has zero offset.
- Accept GNSS fixes from the vessel and maintain a running estimate of the vessel NED position v(t), used as the effective position offset for AIS measurements.
- Compute $h_i(x, t)$ and H_i for any target state x and sensor i, using the appropriate position offset. For AIS, which outputs NED position directly, compute the implied (range, bearing) relative to the vessel and provide the corresponding Jacobian.
- Provide the measurement noise covariance R_i for each sensor, configured from the sensor specifications table.

Correctness of this module is critical to the entire system. Write unit tests verifying that: (a) a known NED-frame target position generates the expected (range, bearing) for each sensor; (b) the measurement function produces the expected output for a known input; and (c) the AIS position-to-observation conversion is consistent with the radar measurement at the same target location.

4.2 Phase 2 — Incremental sensor fusion

Phase 2 adds sensors to the tracker one at a time, always using the NED-frame EKF as the base.

4.2.1 T3: Single-sensor tracker — mm-wave radar

Implement a single-target EKF tracker fed by the mm-wave radar only. Validate with the NIS test and RMSE on Scenario A.

4.2.2 T4: Fusion step 1 — add stereo camera

Extend the tracker to fuse measurements from both the mm-wave radar and the stereo camera. The stereo camera has a different NED position, a restricted 180° FOV, and shorter range. Implement and compare:

- Sequential update: radar update followed by camera update at each scan, using the posterior from the radar step as the prior for the camera step.
- Centralised (joint) update: stack both sensor measurements into a single observation vector and perform one EKF update.

Demonstrate quantitatively which architecture produces lower position RMSE and better NIS consistency and explain the result. Validate on Scenario B.

4.2.3 T5: Fusion step 2 — add AIS

Extend the tracker to additionally incorporate AIS measurements. AIS provides target NED positions at approximately one report per 3s, making this an explicit asynchronous fusion problem. The implementation must handle:

- Asynchronous updates: AIS NED position reports arrive irregularly. Use a time-stamped update queue and trigger an EKF update step (predict to measurement time, then update) whenever an AIS report arrives. The AIS measurement function converts the reported NED position into an implied (range, bearing) relative to the vessel using the GNSS fix closest in time.
- AIS-exclusive targets: targets visible only in AIS must be initiated from AIS measurements alone and tracked at the low AIS update rate.
- AIS dropout: demonstrate that the tracker coasts on radar/camera during the dropout window and smoothly re-acquires AIS when it reappears.

Validate on Scenario C with quantitative comparison of tracking accuracy with and without AIS.

4.3 Phase 3 — Multi-target tracking

4.3.1 T6: Gating and data association

Extend the system from Phase 2 to handle multiple simultaneous targets. Students shall implement:

- Per-sensor, per-track Mahalanobis-distance gating: a detection z_j passes the gate for track i if $d^2(z_j, i) = y^T S^{-1} y \le \gamma$, where $y = z_j - h_i(\hat{x}^-)$ and $S = H_i P^- H_i^T + R_i$. The threshold γ shall be set from the $\chi^2(n_z)$ distribution at gate probability $P_G = 0.99$.
- A sensor availability flag per scan, enabling the system to skip a sensor's contribution in a given cycle without disrupting the overall association.

- A data association algorithm of the group's own choosing, justified by the state-of-the-art survey in T1. The algorithm must resolve the assignment between all active tracks and all gated detections from all sensors simultaneously. Unmatched detections are passed to track initiation; unmatched tracks receive a missed-detection flag.

Validate on Scenario D.

4.3.2 T7: Track management

Implement the full track lifecycle:

- Tentative: a new EKF instance is spawned for each unassigned detection. Initial velocity is estimated from the first two consecutive associated detections via finite difference. Initial covariance reflects position uncertainty from the originating sensor's noise.
- Confirmed: M-of-N confirmation (default M = 3, N = 5, both configurable). Confirmed tracks are reported to the system output.
- Coasting: consecutive missed detections trigger predict-only EKF steps. The covariance grows and the gate widens, improving the chance of re-acquisition.
- Deleted: after K_del consecutive missed detections (configurable). Duplicate tracks are merged using Mahalanobis distance between track state estimates.

Validate on Scenarios D and E using two complementary metrics. **MOTP** (Multiple Object Tracking Precision) measures mean localisation error over all matched track-target pairs and is computed by matching confirmed tracks to true targets via minimum-distance assignment. **Cardinality Error (CE)** measures the mean absolute difference between the number of confirmed tracks and the number of active true targets per scan. Report both metrics as time series and as scalar averages.

4.4 Phase 4 — Real data validation

In the final phase, a real dataset recorded during an experimental campaign in a Danish harbour is to be used. The dataset includes synchronised time-stamped logs from all four sensor nodes: mm-wave radar returns, stereo camera detections, AIS messages, and GNSS fixes from the surface vessel.

Apply the complete tracking system — unchanged from Phase 3 — to the real data and produce:

- Trajectory plots overlaid on a satellite map of the harbour, showing all confirmed tracks and the ground-truth reference positions.
- Quantitative accuracy metrics (RMSE, MOTP, CE) for the tracks where ground truth is available.
- An analysis of discrepancies between simulation and real-data performance, identifying the dominant sources of degradation (model mismatch, unmodelled multipath, sensor calibration errors, etc.).
- At least one concrete proposed improvement motivated by the real-data results.

5 Simulation scenarios

All five scenarios are pre-configured in harbour_simulation.ipynb. The default sensor parameters are: σ_r = 5 m, σ_ϕ = 0.3° (radar); σ_r^c = 8 m, σ_ϕ^c = 0.15° (camera); σ_AIS = 4 m; σ_GNSS = 2 m. Sensor positions in NED: radar at [0, 0] m; stereo camera at [−80, 120] m, boresight 45°. Running each scenario cell produces a JSON file in harbour_sim_output/ that serves as the direct input to the EKF tracker.

5.1.1 Scenario A — Single target, radar only

Purpose: validate single-sensor EKF, coordinate frame transform, and NIS consistency.

- 1 target: CV, initial position [800, 600] m NED, velocity [−2, −1] m/s. Duration: 120 s.
- Radar only active. $P_d = 0.95$, $\lambda_{FA} = 3$.
- Success: track confirmed within 5 scans, RMSE < 12 m (steady state), > 90% NIS within 95% χ^2 bounds.

5.1.2 Scenario B — Single target, radar + stereo camera

Purpose: validate multi-sensor fusion; quantify bearing-accuracy benefit of the camera.

- 1 target crossing the camera FOV ($t \approx 20$–80 s). $P_d = 0.90$ (radar), 0.85 (camera). $\lambda_{FA} = 3, 2$.
- Success: measurable RMSE improvement with camera; NIS consistent for both fusion architectures.

5.1.3 Scenario C — AIS-equipped target, asynchronous fusion, AIS dropout

Purpose: validate asynchronous AIS fusion and coasting through a 30 s AIS blackout.

- 1 AIS-equipped target. All sensors active. AIS dropout: $t = 60$–90 s. Duration: 150 s.
- Success: track survives dropout; RMSE lower with AIS during available windows; smooth re-acquisition.

5.1.4 Scenario D — Multiple targets, crossing trajectories

Purpose: validate gating, data association under conflict, and track identity preservation.

- 4 targets on converging courses, crossing within ~30 m near $t = 50$–70 s. $P_d = 0.88$, $\lambda_{FA} = 5$.
- Success: all 4 tracks maintained; no identity swap; MOTP < 15 m and CE < 0.5.

5.1.5 Scenario E — Mixed AIS / non-AIS harbour traffic

Purpose: validate the complete multi-target pipeline with dynamic scene cardinality.

- 6 targets (3 AIS-equipped, 3 non-AIS). Targets enter and leave the scene. Duration: 180 s.
- $P_d = 0.90$ (radar), 0.85 (camera), 0.98 (AIS). $\lambda_{FA} = 6$ (radar), 3 (camera).
- Success: all 6 targets tracked; MOTP < 20 m and CE < 1.0.

6 Deliverables

The project deliverables are D1 Software repository and D2 Technical poster. Both deliverables must be delivered by the group by **May 8 2026 at 17:00 (CET) in DTU Learn**.

6.1 D1 — Software repository

A documented, reproducible Python repository.

6.2 D2 — Technical poster

A single A1 landscape poster (provided template) structured in five panels:

1. Background and motivation: the harbour surveillance problem, why multi-sensor fusion is necessary, and a concise summary of the state-of-the-art including the justification for the chosen data association algorithm.
2. System architecture: block diagram of the full system, sensor specifications table, NED coordinate frame description, and measurement model Jacobian summary.
3. Implementation highlights: the EKF measurement model with sensor-position-dependent Jacobians, the chosen fusion architecture, the selected data association algorithm and its properties, and the track lifecycle.
4. Results: the most significant quantitative results from both simulation and real data.
5. Conclusions and future work: what the system can and cannot do, the most important lessons from real-data validation, and one concrete direction for improvement.